Содержание

Введение 4
1 Вводная часть 6
1.1 Интернет вещей 6
1.2 Почему облако? 9
1.3 SAP HANA 11
1.4 Архитектура рассматриваемой ГПС 14
2 Формулировка создания интерфейса 17
2.1 Основы создания пользовательского интерфейса 17
2.2 Используемые виды диалога при решении задач ГПС 18
2.3 Построение схемы использования интерфейса пользователем 20
3 Анализ основных инструментов создания интерфейса 22
3.1 SAPUI5 22
3.2 SAP Fiori 22
3.3 Использование сторонних библиотек 23
3.4 Использованные инструменты 24
4 Разработка интерфейса диалога 25
4.1 Составные части интерфейса 25
4.2 Создание интерфейса управления процессом 27
4.3 Создание интерфейса управления складом 30
Заключение 32
Список литературы 33

Введение

Большая часть задач, связанных с оперативным управлением ГПС, начиная от представления исходных данных и формализации модели ГПС и кончая выработкой управляющих команд, непосредственно связана с интеллектуальной деятельностью человека и их решения базируются на его интуиции, опыте, образном мышлении и т. п. Перспективным путем решения слабо структурированных и трудноформализуемых задач в системах управления является непосредственное обеспечение взаимодействия человека-оператора и системы управления ГПС.

Такое взаимодействие представляет собой совокупность диалоговых процедур, рассматриваемых как способ совместного решения задач, отличающийся от автономной работы ЭВМ или человека и позволяющий учитывать трудноформализуемые факторы и условия ГПС, а также ее окружения; применять эвристические методы для решения задач, не поддающихся решению точными методами или требующих слишком большого времени для решения; постепенно наращивать систему управления, на первых этапах передав большую часть функций на выполнение оператору с дальнейшим постепенным переходом к оптимальному их разделению между человеком и ЭВМ; выбирать лучшее решение из конкурентоспособных за счет интеллекта человека-оператора (эксперта).

Нереальность создания в настоящее время полностью автоматических систем управления для ГПС из-за недостаточного развития вычислительной техники, математических методов и т. п. не является единственной причиной наблюдаемого быстрого развития диалоговых систем управления. Дело в том, что полностью автоматическая система не представляется идеальной и для ближайшего будущего в развитии ГПС и ИАСУ, Это вызвано следующими причинами:

сложностью производственного процесса в ГПС с большим числом всевозможных комбинаций и вариантов решений;

возможностью возникновения ситуаций, не предвиденных разработчиками ИАСУ, что является типичным для сложных систем;

неполной автоматизацией производственного процесса, требующей участия человека в выполнении ряда обслуживающих операций, наладки, ремонта и др.;

диспетчированием в реальном масштабе времени, ставящим новые задачи, решение которых в теории управления в настоящее время отсутствует либо имеется для частных случаев;

недостаточностью мощности вычислительных средств и невозможностью формализованного учета всех факторов ГПС при управлении;

Знания оператора ГПС об объекте управления — его свойствах, характеристиках и возможностях, а также накопленный опыт управления

позволяют осуществить приближенную оценку ситуаций, сложившихся в ГПС, и прогнозировать их дальнейшее развитие. В настоящее время человек способен делать это быстрее и лучше ЭВМ. Получение таких оценок и доступность их УВК ГПС через диалог позволяет повысить качество принятия решений в ИАСУ и увеличить быстродействие. [1]

На сегодняшний день в многих странах все больше внимания уделяют на облачные технологии. На производстве развивается два подхода к адаптации этих технологий: непосредственное внедрение облачных вычислений в информационную инфраструктуру предприятия, а также создание «облачного производства».

Чем же так называемые «облака» заслужили столь пристальное внимание? Прежде всего, это концепция предоставления требуемых вычислительных ресурсов в нужное время в нужном месте с высокой степенью надежности и масштабируемости. Ключевые понятия облачных вычислений – это «что угодно как услуга» (Everything as a Service, XaaS), куда входят «программное обеспечение как услуга» (SaaS), «платформа как услуга» (PaaS) и «инфраструктура как услуга» (IaaS).

Преимущества этих услуг (сервисов) – на лицо: вычислительные ресурсы любой мощности, масштабов и возможностей пользователь может получить в желаемый момент времени в рамках модели повременной оплаты без необходимости покупки, установки и обслуживания дорогостоящего оборудования, найма дополнительных специалистов и т.д. Любую вычислительную задачу теперь представляется возможным решить в «облаках», достаточно иметь под рукой PC, Mac или портативный компьютер и доступ к сети Интернет. В таком случае реализация основной функциональности диалога пользователя со системой управления ГПС перейдет на интернет-технологии, которые развиваются в довольно быстром темпе. [3]

В данном курсовом проекте будет рассмотрена диалог пользователя со системой управления ГПС на основе облачной технологии компании SAP, на облачной платформе HANA Cloud Platform. Будут проанализированы возможные пути решения создания интерфейса и формулировка решения по выбранному пути. Главной задачей проекта является разработка интерфейса системы управления ГПС.

1 Вводная часть

1.1 Интернет вещей

Интернет вещей — концепция межсетевого взаимодействия физических предметов («вещей»), оснащённых встроенными технологиями (сенсорами, датчиками и т.д.) для взаимодействия друг с другом или с внешней средой.

Идеология интернета вещей направлена на повышение эффективности экономики за счет автоматизации процессов в различных сферах деятельности и исключения из них человека.

На начало 2016 года в использовании технологий интернета вещей компании ориентируются в первую очередь на массовые сегменты IoT, где побуждением конечных пользователей к использованию решений и сервисов IoT являются рыночные стимулы, такие как:

1. «Умный дом»:
2. «Умный транспорт», включая:
3. Торговля и финансовые услуги:
4. Промышленный сегмент – перевод АСУТП на принципы IoT.

Общепринятое определение, что же такое Промышленный интернет вещей, пока не сформировалось. В этом контексте следует отметить две инициативы, которые во многом определяют два подхода: европейский и американский.

Европейский подход базируется на родившейся в Германии концепции «Индустрия 4.0» (Industry 4.0). Согласно Industry 4.0, европейская экономика находится на пороге четвертой промышленной революции. Четвертая революция — «умные» предприятия — на самом деле является продолжением третьей. Ее знаковые моменты — внедрение киберфизических систем и переход к персонализированному производству.

В промышленном IoT основными разновидностями «вещей», которые надо подключать к сети, являются различные типы датчиков (сенсоров) и приводов. Эти устройства с одной стороны имеют интерфейс с коммуникационной сетью, а с другой — интерфейс, обеспечивающий физическое взаимодействие с процессом, который требуется отслеживать. Задача датчиков и сенсоров — сбор информации.

Коммуникационный интерфейс — совершенно необходимый компонент устройства IoT. Это может быть проводной или беспроводной интерфейс.

Как устроены современные системы промышленной автоматизации? Их можно разделить на несколько уровней (см. рис. 1). Непосредственно «в поле», например в цехе предприятия, располагаются различные сенсоры, датчики и приводы. Далее следует уровень контроля — например, программируемые логические контроллеры, которые, собственно, собирают информацию с датчиков и управляют приводами. Уровень управления

процессами формируют системы класса SCADA (Supervisory Control And Data Acquisition) и MES (Manufacturing Execution System). Наконец, на вершине пирамиды — системы планирования ресурсов предприятия (Enterprise Resource Planning, ERP), которые, как правило, работают на серверах, расположенных в корпоративных центрах обработки данных (ЦОДы).

Рис. 1 Переход от классических АСУ ТП к Промышленному интернету вещей

Что изменится в этой структуре с реализацией концепции Industry 4.0? Нижний уровень (с датчиками, приводами и прочими исполнительными механизмами) сохранится, однако число устройств на этом уровне экспоненциально вырастет. Кроме того, что даже более важно, устройства этого уровня будут наделяться все большим интеллектом. Они станут частью киберфизических систем и будут способны автономно выполнять многие функции. Большинство же функций, которые в сегодняшних системах реализуются устройствами вышестоящих уровней, будут переноситься на высокопроизводительные серверы, которые будут располагаться в серверных кластерах, ЦОДах или облаках.

Преимущества новой структуры в том, что общее количество управляющих систем сократится, что упростит сам процесс управления. Кроме того, эффективность использования ресурсов повысится, а средств потребуется меньше.

Массовая реализация описанного выше подхода в системах промышленной автоматизации пока еще тормозится рядом нерешенных проблем. Большая их часть связана именно с сетевой инфраструктурой: низкой производительностью передачи данных, недостаточной надежностью, непредсказуемыми задержками между устройствами полевого уровня и обслуживающими их серверами. Однако на решение этих проблем брошены лучшие силы многих производителей, что внушает оптимизм.

Как уже говорилось, количество подключенных устройств на «умных» фабриках будущего станет значительно больше, чем на современных производствах. Объемы собираемых данных также кардинально увеличатся. Поэтому необходимы технические решения, способные обеспечить подключение большого числа различных устройств просто и эффективно, при выполнении требований к производительности, безопасности и надежности.

Другой важный момент — это снижение размеров устройств и сокращение энергопотребления. Прогресс в области полупроводниковой техники позволяет производить все более компактные структуры, обеспечивать более высокую степень интеграции и сокращать энергопотребление.

Для многих применений в промышленности необходима гарантированная задержка при передаче данных. Причем такие гарантии могут потребоваться не только при связи объектов в пределах производственной зоны или предприятия, но и при взаимодействии с объектами вне предприятия. В настоящее время существует ряд протоколов реального времени, которые способны обеспечить жесткие гарантии по задержке в сети Ethernet. Но ни один из этих протоколов не является стандартом Ethernet.

Для работы в режиме реального времени могут использоваться различные технологии — например, протокол Precision Time Protocol (PTP), который обеспечивает синхронизацию часов, встроенных в сетевые устройства. Этот протокол уже активно применяется во многих сетях. Организация IEEE постоянно работает над совершенствованием PTP, в 2016 году должен быть принят стандарт уже на третью версию этого протокола. Достижению стабильной низкой задержки способствуют также повышение пропускной способности каналов связи и применение алгоритмов приоритизации трафика внутри коммутаторов. Очевидно, что чем более широкая полоса пропускания доступна, тем ниже вероятность того, что коммутатор блокирует тот или иной пакет.

С увеличением числа подключенных к сети устройств и повышением значимости ее бесперебойной работы кардинально меняются требования к ее администрированию и эксплуатации. До сих пор огромный объем связанных с сетями работ выполняется вручную. Проектирование сети, инсталляция оборудования, его конфигурирование, тестирование, мониторинг работы, техническое обслуживание, поиск и устранение неисправностей — все это требует огромных человеческих и финансовых ресурсов. В будущих сетях IoT доля ручного труда должна существенно сократиться. Это необходимо, в частности, потому, что при увеличении числа подключенных устройств в сотни и тысячи раз станет физически невозможно, например, настраивать каждое отдельное устройство.

Средства автоматического конфигурирования сетевого оборудования, управления сетью и пр. активно развиваются. Они не только экономят время

и ресурсы, но и позволяют существенно снизить вероятность ошибок, связанных с человеческим фактором. Однако работы здесь еще непочатый край.

Ключевое значение имеют вопросы обеспечения безопасности таких решений. Отказоустойчивость сетей — также чрезвычайно важный аспект. Жесткие условия эксплуатации повышают вероятность повреждения тех или иных элементов сетевых структур, при этом последствия простоя промышленной сети могут иметь огромный негативный эффект. В некоторых случаях такой простой вообще недопустим, потому что может привести к катастрофам и массовой гибели людей. На данный момент существует немало механизмов, обеспечивающих горячее резервирование сетевых элементов и гарантирующих продолжение работы даже в случае повреждения части узлов и каналов связи. Такие механизмы активно развиваются и должны обязательно использоваться в промышленных сетях.

На какой стадии своего развития сегодня находится Промышленный интернет вещей? Безусловно, он уже существует в той или иной мере — в виде систем автоматизации производства. На многих предприятиях множество устройств подключены к сетям промышленных зон, взаимодействие между вещами осуществляется и на дальних расстояниях. Но это только начало. Как уже говорилось, с увеличением количества подключенных устройств, реализацией киберфизических систем Промышленный интернет вещей выйдет на новый уровень

Что касается инфраструктурных решений, многие технологии для надежного и эффективного взаимодействия промышленных «вещей» также уже существуют. Однако немало и пробелов, которые необходимо восполнить. Высокий интерес к технологии IoT со стороны большого количества крупных компаний, свидетельствует о том, что существующие проблемы будут решены в ближайшем будущем и IoT в скором времени будет все активнее внедряться на предприятиях.

1.2 Почему облако?

Для ответа на этот вопрос мы попробуем разобраться в преимуществах и недостатках данной технологии.

Преимущества:

Множество ИТ экспертов полагают, что в недалеком будущем облачные вычисления войдут в топ-список технологических трендов. Во всяком случае, для такого смелого утверждения должны существовать определенные причины, почему именно cloud computing вскоре выберут множество Интернет-бизнесов. Таким образом, попытаемся проанализировать преимущества облачных технологий.

Во-первых, это высокий уровень виртуализации. Как уже упоминалось выше, пользователь, развернув свое приложение, не задумывается о таких

проблемах, как аппаратные средства, их поддержка, конкретный софт, установленный на машине. Он даже не задумывается о том, на какой именно или даже на каких именно машинах исполняется его приложение.

Возможность запуска множество копий приложения на многих виртуальных машинах представляет преимущества масштабируемости: количество экземпляров приложения способно практически мгновенно увеличиваться по требованию, в зависимости от нагрузок.

И, наконец, датацентры управляются профессиональными специалистами, обеспечивающими круглосуточную поддержку функционирования виртуальных машин. И даже если физическая машина «рухнет», благодаря распределению приложения на множество копий оно все равно продолжит свою работу. Это создает определенный высокий уровень надежности и отказоустойчивости функционирования системы.

Однако следует заметить, что в глазах потребителей сервиса основным преимуществом облачных вычислений является, пожалуй, отсутствие необходимости закупать все соответствующее оборудование и ПО, а затем поддерживать их работу. Этот аспект мы рассмотрим чуть позже в связке с анализом применения в области Интернет-стартапов.

Таким образом, все эти преимущества определяют выбор облачных вычислений по сравнению с другими схожими технологиями. Именно они «толкают» на дальнейшие разработки крупнейшие ИТ корпорации, которые так же видят повышенный интерес у пользователей системы. Скорее всего, уже в ближайшем будущем мы увидим если и не всплеск научной и практической активности в сфере cloud computing, то уж точно постепенное увеличение определенной заинтересованности.

Недостатки:

Казалось бы, преимущества облачных вычислений настолько очевидны, что все компании в скором будущем массово перейдут на использование этого вида технологии. Однако существуют некоторые проблемы, с которыми сталкиваются как вендоры cloud computing, так и бизнес-пользователи.

Во-первых, как упоминается в статье «Встречный план» Дмитрия Петрова, практически отсутствует определенная культура потребления аутсорсинговых услуг, бизнес относится к такого рода услугам несколько настороженно. «Причин же недоверчивого отношения малого и среднего бизнеса к дата-центрам может быть несколько. Скорее всего, это боязнь лишиться контроля над ИТ-ресурсами, опасения насчет гарантии сохранности и защиты переданной информации и представление дата-центра лишь как площадки для размещения оборудования».

Таким образом, вендорам сервисов облачных вычислений следует понимать важность создания у клиента осведомленности об услуге, а так же чувства безопасности собственных данных.

Во-вторых, некоторые эксперты, например Г. Маклеод (Hugh Macleod) в статье «Самый хорошо охраняемый секрет Облаков», утверждают, что

облачные вычисления ведут к созданию огромной, невиданной ранее монополии. Возможно ли это? Конечно, на рынке облачных вычислений для помещения в облако какой-либо информации, в отношении которой существуют правила информационной безопасности, компании будут скорее использовать таких вендоров, чье имя «на слуху» и кому они доверяют. Таким образом, существует определенная опасность того, что все вычисления и данные будут агрегированы в руках одной сверхмонополии. Однако на данный момент на рынке уже существуют несколько компаний с примерно одинаковым высоким уровнем доверия со стороны клиентов (Microsoft, Google, Amazon), и нет никаких фактов, которые бы указывали на возможность доминирования одной компанией всех остальных. Поэтому в ближайшем будущем появление глобальной сверхкомпании, которая будет координировать и контролировать все вычисления в мире, очень маловероятно, хотя одна лишь возможность такого события отпугивает некоторых клиентов.

В-третьих, государство, на территории которого размещен датацентр, может получить доступ к любой информации, которая в нем хранится. Например, по законам США, где находится самое большое количество датацентров, в этом случае компания-провайдер даже не имеет права разглашать факт передачи конфиденциальной информации кому-либо, кроме своих адвокатов.

Эта проблема является, наверное, одной из самых существенных в вопросе вывода конфиденциальной информации в облако. Путей ее решения может быть несколько. Во-первых, можно шифровать всю информацию, помещаемую на облако. Во-вторых, можно просто ее туда не помещать. Однако, во всяком случае, у компаний, пользующихся облачными вычислениями, это должно быть определенным пунктом в списке вопросов информационной безопасности. Кроме того, сами провайдеры должны улучшать свои технологии, предоставляя некоторые услуги по шифрованию.

Таким образом, определенные проблемы существуют, но практически все они сконцентрированы в области неправильного или неадекватного понимания технологии и возможностей ее использования. И лишь проблема неограниченного доступа государства к информации, хранящейся в датацентре, на мой взгляд, является на данный момент самой труднорешаемой и самой ограничивающей круг пользователей технологией cloud computing.

1.3 SAP HANA

SAP HANA — это высокопроизводительная NewSQL платформа для хранения и обработки данных, в основе которой лежит технология вычислений in-memory с использованием принципа поколоночного хранения данных, платформы, разработанной и выведенной на рынок компанией SAP

SE. Архитектура HANA обеспечивает как высокоскоростную обработку транзакций, так и работу со сложными аналитическими запросами, совмещая решение этих задач в рамках единой платформы.

HANA IoT предоставляет интерфейсы для регистрации устройств и их конкретных типов данных, отправки этих данных в базу данных, работающих на SAP HANA Cloud Platform.

Данная платформа содержит две основные компоненты: Дистанционное обслуживание и Управление устройствами (RDMS) и служба управления сообщениями (MMS). Кроме того, есть графический пользовательский интерфейс Internet of Things Services cockpit, которая обеспечивает легкий доступ к различным службам (рис. 2). [4]

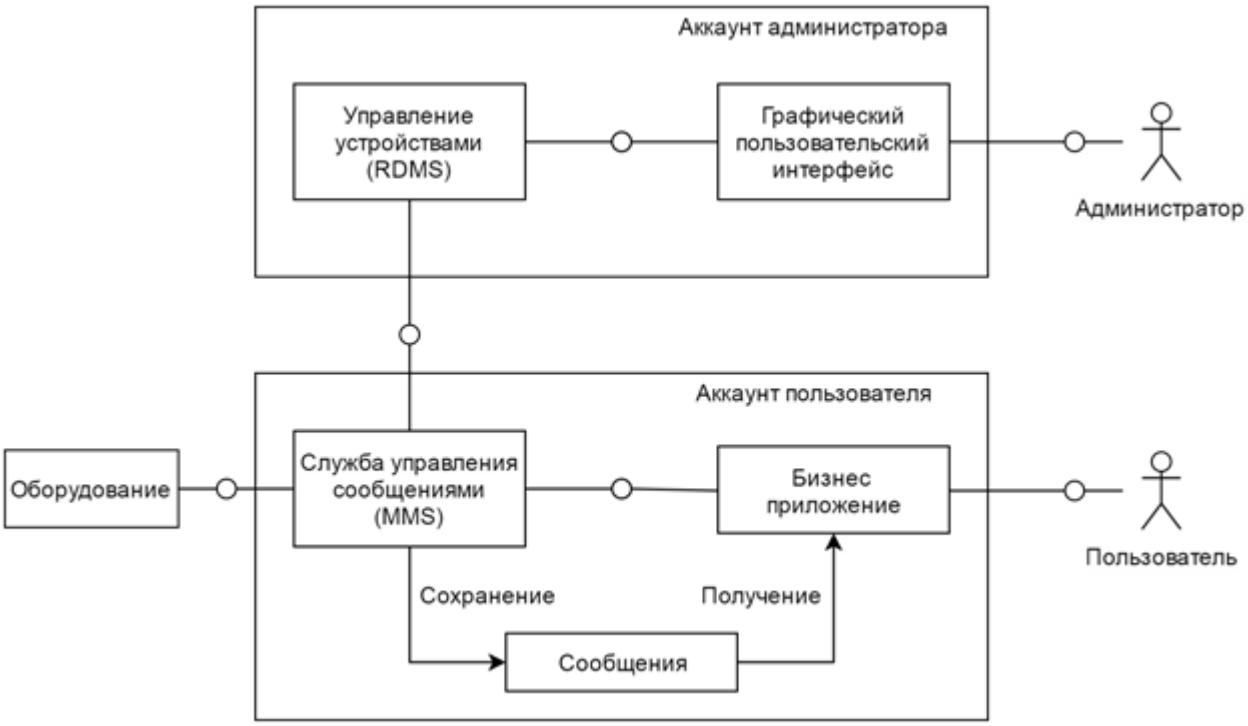


Рис 2. Архитектура SAP HCP IoT service

Платформа поддерживает несколько языков и библитотек разработки, таких как JAVA, HANA XS, HTML5, SAPUI5.

SAP HANA XS включает в себя полнофункциональный сервер приложений, который основан на базах данных SAP HANA. Этот сервер приложений может быть использован для разработки веб-приложения, на основе технологий, SAPUI5, серверного JavaScript, и нативных объектов SAP HANA (например, SQLScript процедур). Дает возможность обращаться к серверу с http-запросом. Делает архитектуру всего приложения более простым за счет отделения части представления от части обработки данных. SAP HANA XS является не обязательным компонентом.

Типовой проект SAP HANA XS развернутого на SAP HCP показан на рисунке 3.

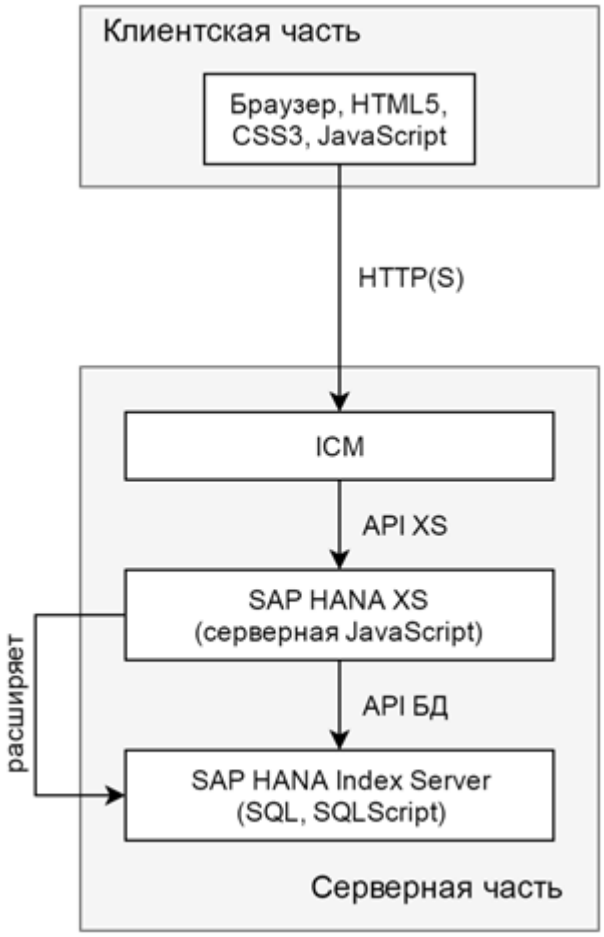


Рис 3. Схема клиент-серверного взаимодействия HANA XS

Клиентская часть приложения основана на фреймворке SAPUI5. Составляющие библиотеки обрабатываются с помощью движка HANA которой включает в себя встроенный веб сервер.

На серверной стороне приложение использует две системы: HANA DB (Index Server) и HANA XS. Задачи распределяются между ними. HANA DB в виде СУБД отвечает только за удаление, перемещение данных и отображение. БД хранит данные на оперативной памяти. В случае более сложной логики требуется язык прототипно-ориентированный язык, например Javascript который поддерживает HANA XS.

СУБД SAP HANA состоит из нескольких серверов и главного сервера индексации. Основные процессы связанные с данными производятся на сервере индексации. Этот же сервер хранит в себе актуальные данные и движки для предоставления данных, обрабатывает входящие SQL и MDX запросы (рис. 4). [6]

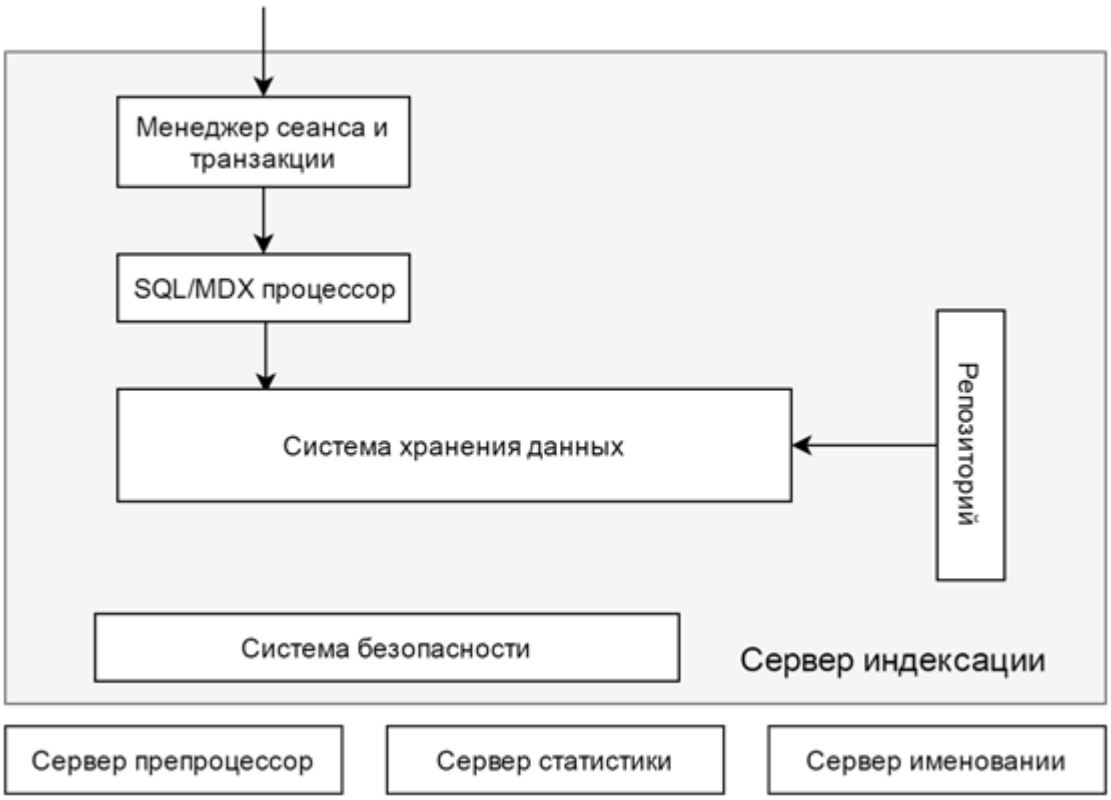


Рис. 4 Архитектура SAP HANA

Каналы связи между клиентским уровнем и серверной SAP HANA XS в первую очередь определяются с использованием протокола OData. Этот протокол основан на технологии REST который используется в SAPUI5 и JavaScript-виджетах.

1.4 Архитектура рассматриваемой ГПС

ГПС состоит из двух роботов-манипуляторов RV-2AJ компании Mitsubishi Electric токарного и фрезерного станка, конвейера, склада и системы видеонаблюдения (рис. 5).

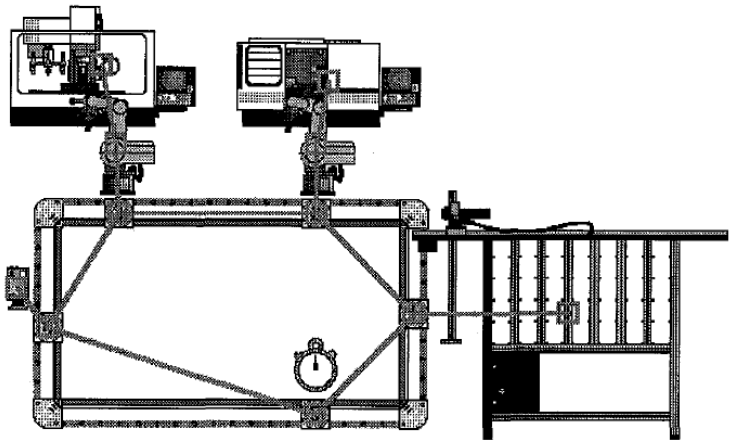


Рис. 5 Состав ГПС

Каждое оборудование управляется из облака с помощью одноплатного компьютера Raspberry Pi. На платах Raspberry Pi находится скрипт на языке программирования Python, который автоматически запускается при их включении в сеть (рис. 6).

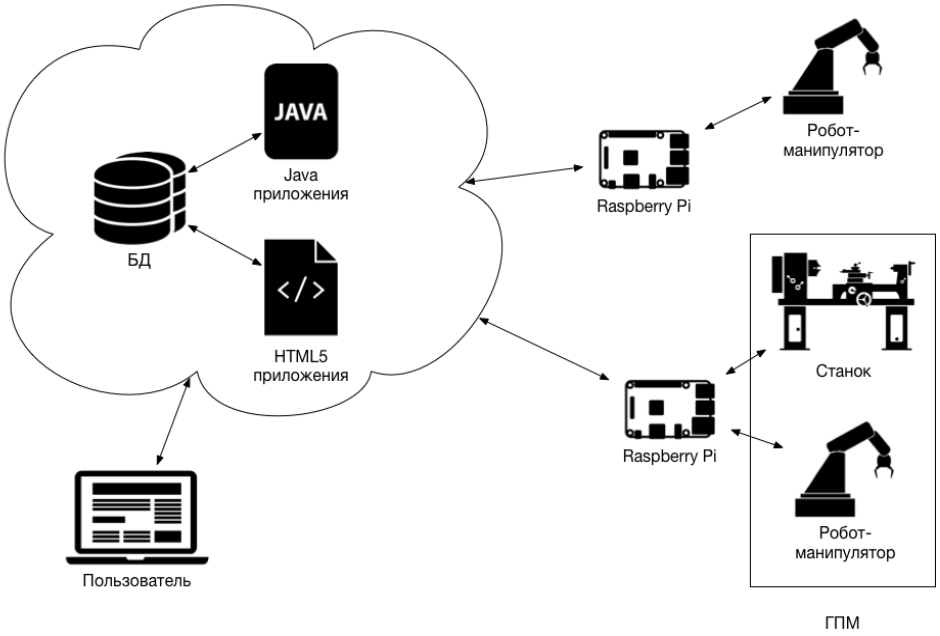
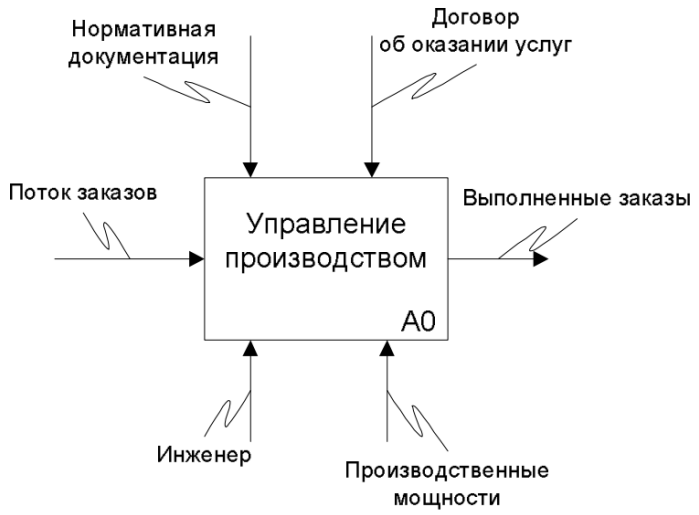


Рис. 6 Структура системы управления ГПС

Функциональная модель процесса управления ГПС-ом показана на рисунке 7. Процесс имеет итеративный характер. С помощью обмена данных происходит определение следующей операции, выполнение, получение подтверждения и т.д.



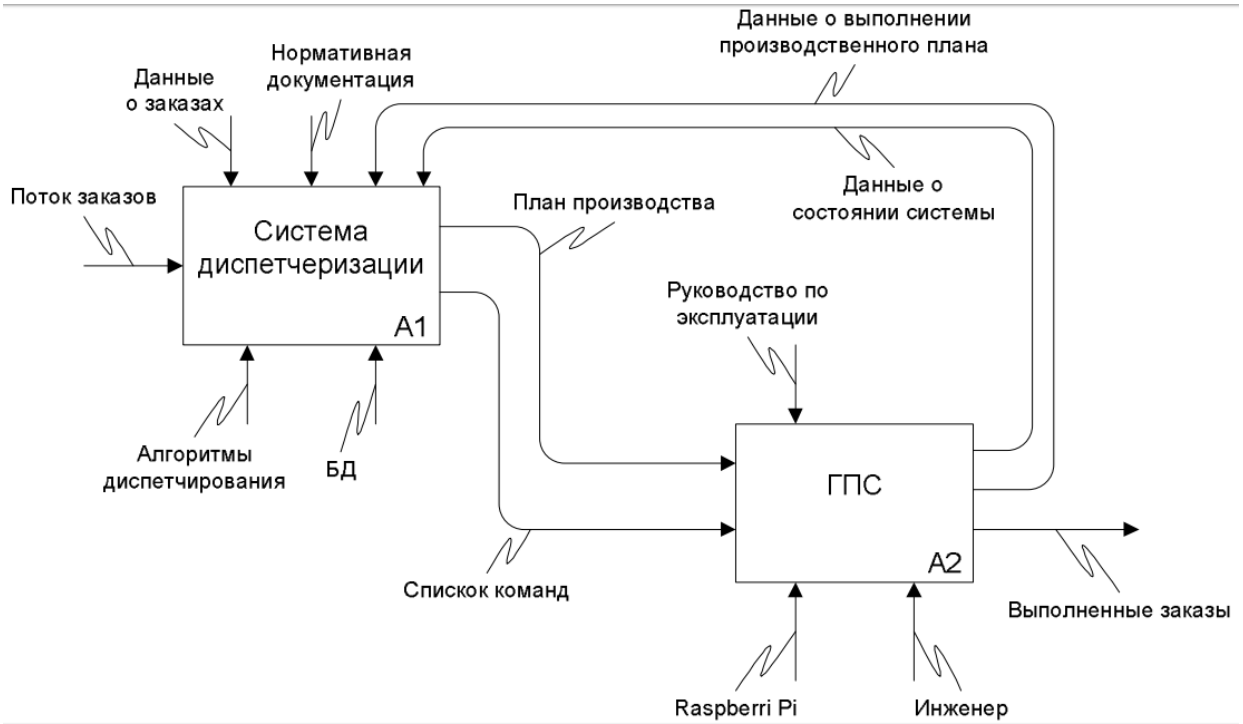


Рис. 7 Функциональная модель

2 Формулировка создания интерфейса

2.1 Основы создания пользовательского интерфейса

Интерфейс - система правил и средств, регламентирующая и обеспечивающая взаимодействие нескольких процессов или объектов.

Пользовательский интерфейс (ПИ) - система правил и средств, регламентирующая и обеспечивающая взаимодействие программы с пользователем. В понятие пользовательского интерфейса (ПИ) входит не только, и даже не столько, картинка на экране - трехмерная, анимированная или просто выполненная в модном дизайне, а способы взаимодействия пользователя с системой.

Отправной точкой хорошего интерфейса является метафора. Обстановка на экране и способы взаимодействия с системой должны апеллировать к ситуации, хорошо знакомой пользователю. Так, оконный интерфейс задумывался как метафора рабочего стола с документами.

В рамках метафоры разрабатывается концептуальный дизайн, своего рода алфавит взаимодействия, изучив который, пользователь сможет легко делать то, что ему нужно. Еще необходимо найти изящный способ изображения, как отдельных элементов, так и их групп.

Концептуальный дизайн интерфейса должен базироваться на идее интерфейсной среды. На время работы с системой пользователь погружается в среду интерфейса. Слово среда применяется как обозначение типичной для поведения человека в различных средах связки «сигнал - действие».

Понятия среды и понятие метафоры близко связаны. Если среда по виду и некоторым опорным элементам будет напоминать пользователю что-то уже знакомое, он сможет быстрее приспособиться к ней. Вместе с тем выбранная метафора может продиктовать все изобразительные решения дизайна интерфейса. Однако следует остерегаться фотографической схожести среды в компьютере с выбранной метафорой. Все-таки компьютерная среда искусственная, и полностью повторить все элементы взаимодействия из физического мира не удастся.

Мы подходим к важному принципу построения дизайна интерфейса - балансу между интерактивными возможностями программы и сложностью ее изобразительного ряда. В интерфейсе должен обеспечиваться баланс между функциональными возможностями программы, возможностями манипуляции ею и ее изобразительным рядом.

Создавая интерфейс, надо закладывать систему динамических визуальных сигналов с самого начала, поскольку они являются столь же естественной, сколь и необходимой частью анимационного интерфейса.

Кроме того, информационная емкость (т.е. количество разных различимых вариаций) динамических сигналов огромна. Современные дисплеи отображают миллионы цветов, но даже если человеческий глаз и в состоянии отличить столько оттенков, человеческий мозг не в состоянии

придавать им смысл. С другой стороны, такой простой сигнал, как мигание, имеет действительно миллионы хорошо осознаваемых оттенков, связанных с изменением яркости объекта во времени.[2]

Для использования анимационного интерфейса придется переходить к программам, управляемым временем. Вне зависимости от активности пользователя программе, построенной на анимационном интерфейсе, всегда есть что делать (например, менять фазу мигания). При этом, естественно, она должна постоянно быть доступной для взаимодействия, но, в отличие от многих сегодняшних мультимедиа-программ, не прерывать отображаемый поток, а плавно изменять его в соответствии с воздействием пользователя.

Такие требования легче всего реализуются в специфической архитектуре программ, управляемых временем. На каждом такте работы такой программы заново строится изображение на экране, а события, инициированные пользователем, например ввод с клавиатуры, отрабатываются всего лишь изменением состояния программы. Соответствующее изменение на экране происходит (быть может, не сразу) на очередном временном такте. Таким образом, к двум привычным уровням программы - функциональному и интерфейсному - добавляется визуальный.

Для дизайна конкретной программы требуется разработка собственной среды взаимодействия (направленной на реализацию конкретной функциональности) на базе общепринятой системы динамических визуальных сигналов. Предпочтительно иметь сквозное визуальное решение.

После выработки сквозного визуального решения необходимо прорисовать картинки, называемые у аниматоров фонами. Точнее назвать их *неподвижной составляющей подвижного изображения*. На каждом фоне надо расположить анимированные элементы взаимодействия. И, наконец, самое трудное - надо спроектировать визуальные переходы между существенно отличающимися состояниями. [2]

2.2 Используемые виды диалога при решении задач ГПС

Знания оператора ГПС об объекте управления — его свойствах, характеристиках и возможностях, а также накопленный опыт управления позволяют осуществить приближенную оценку ситуаций, сложившихся в ГПС, и прогнозировать их дальнейшее развитие. Получение таких оценок и доступность их через диалог позволяет повысить качество принятия решений и увеличить быстродействие. Здесь имеют место две стороны диалога:

1) определение, а в ряде случаев уточнение состояния ГПС и прогноз развития этого состояния на ближайшее будущее;

2) формирование решений по управлению.

Большинство задач ГПС так или иначе включают диалоговые процедуры. Поэтому в СУ можно выделить несколько видов диалога (рис. 8), относящихся к различным функциональным подсистемам, СУБД и др.

В СУ ГПС необходима единая система диалоговых процедур, позволяющая на общих принципах и подходах реализовать решение различных задач, стоящих перед управленческим персоналом ГПС. В этой системе можно выделить три группы процедур.

Первая группа диалоговых процедур в СУ связана с этапами генерации системы, ее настройкой или адаптацией к изменяющимся условиям производства. С их помощью осуществляется организация файловой структуры БД, генерация ПМО и т. п. Диалог при выполнении указанных процедур выполняется с использованием стандартных процедур СУБД SAP HANA.

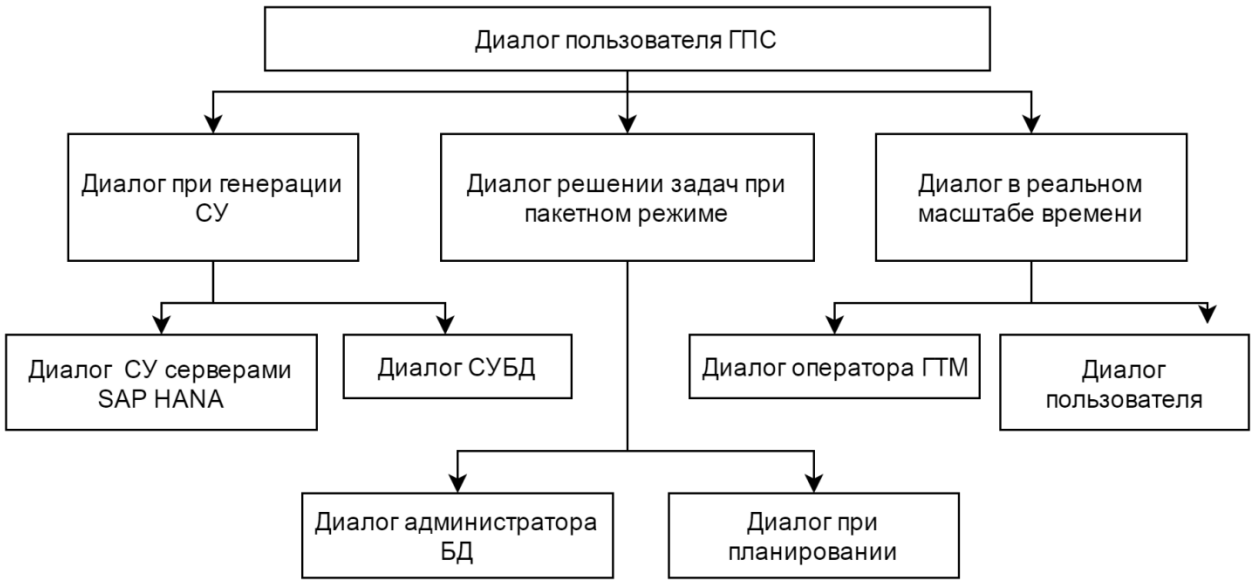


Рис. 8 Виды диалогов

Вторая группа диалоговых процедур связана с решением в пакетном режиме задач, выполняемых в процессе функционирования СУ. Это, прежде всего, задачи планирования ПП в ГПС.

Оператор СУ осуществляет ввод исходных и недостающих данных, выбирает алгоритм составления расписания (приоритетные правила), непосредственно оказывает влияние на ветвление программы, выбор варианта решения и т. п. Диалог администратора БД в данной группе процедур сводится к корректировке файловой структуры, ведению журнала БД, контролю за работой пользователей, восстановлению базы при сбоях и т. п.

Наконец, третья группа диалоговых процедур в СУ связана с диспетчированием. При этом решаются задачи контроля хода ПП, его диспетчирования, прогнозирования, обновления информационной базы, ввода и вывода текущей и итоговой информации и ряд других. Решение

указанных задач связано непосредственно с ПП и поэтому имеет решающее значение для функционирования ГПС.

Требования для диалоговой процедуры:

1. Оптимальное сочетание информации о ходе ПП и состоянии элементов ГПС как в агрегированном виде, так и в детализированном.
2. Вывод информации должен осуществляться в наглядной и удобной для оператора форме.
3. Максимальное облегчение ответа оператора на вопросы ЭВМ — использование подсказок, вариантов ответов, режима меню, форматов для ввода ответов и т. д.
4. Обеспечение работы системы при разной квалификации оператора. Это возможно при условии наличия диалога с разной степенью детализации выводимой и вводимой информации.
5. Право на ошибку. Система при ошибочном ответе должна предоставлять оператору возможность быстрого ее обнаружения и исправления. Следовательно, должен быть предусмотрен механизм проверки ответов.
6. Предоставление пользователю информации о восприятии ЭВМ его ответа, причем это должно делаться с высокой оперативностью.

Под управляющей командой понимается партия информации, передаваемый в XS Server, в котором производится синтаксическая проверка на соответствие типа переменной и формата. При диалоге предусмотрена возможность повторного ввода при выявлении ошибки. Семантическая проверка осуществляется на нахождение значений параметров команды в области их определения, на верность логических связей между параметрами и на возможность или невозможность выполнения команды.

При вводе команд для ГТМ проверяется возможность выполнения вводимой команды: соответствие состояния ресурсов, указанных в команде, их допустимым значениям для данной операции. Например, для операции обработки проверяется наличие передаточной партии заготовок на входном порту ГТМ; работоспособность и наладка станка; стойкость комплекта инструмента.

Если состояние ресурсов соответствует допустимому, вводимая команда обрабатывается и передается на модуль, если какой-либо ресурс отсутствует, выдается ответ оператору.[1]

2.3 Построение схемы использования интерфейса пользователем

Здесь будет описана схема окон пользовательского интерфейса, который имеет вид показанный на рисунке 9.

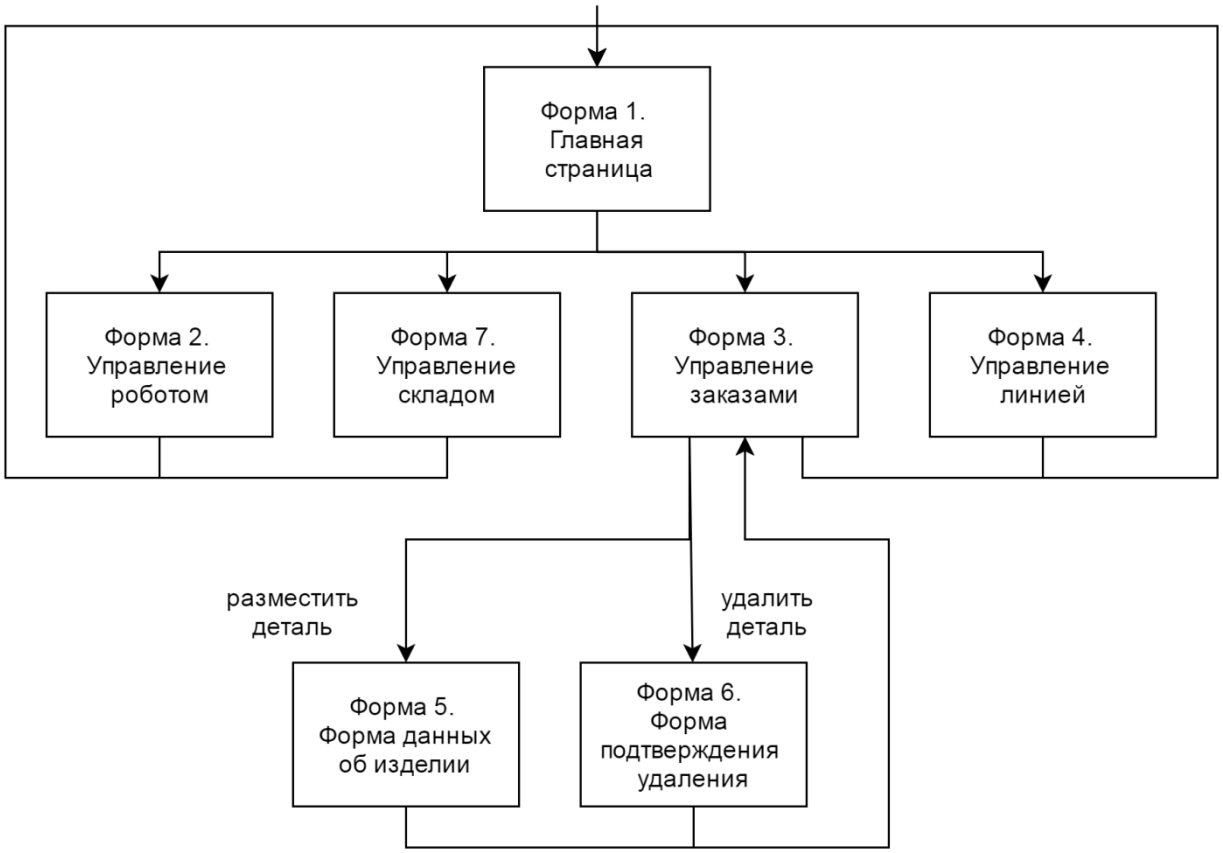


Рис. 9 Формы пользовательского интерфейса

При первой загрузке системы управления пользователь попадает на форму 1, в которой с помощью меню можно перемещаться между фреймами «Управление роботом», «Управления заказами», «Управление линией».

Форма 2 представляет собой рабочий стол управления роботом, которая состоит из

- окна управления роботом, при помощи точек, означающих координаты на одной поверхности
- окна управления роботом, с заданием направления
- окна показывающей текущие результаты операции.

На форме 3 можно увидеть диаграмму Ганта. Также на этой форме можно добавлять новое изделие или удалять существующее, которые в свою очередь вызывают 5-ую и 6-ую форму соответственно.

С помощью формы 4 ведется управление линией. К управлению линией входят такие функции как: инициализация, подать роботу, взять заготовку, подать в станок, обработать, вернуть на конвейер, отправить на склад. Также можно наблюдать за ходом производственного процесса на заводе через видеокамеру.

Форма 7 отвечает за управление складом и дает такие функции как «Аварийная остановка», «Забрать с робокара», «Поставить на конвейер», «Взять с конвейера», «Взять любую заготовку», «Поставить в свободную ячейку».

3 Анализ основных инструментов создания интерфейса

3.1 Средство создания интерфейса SAPUI5

SAPUI5 это технология пользовательского интерфейса клиента на основе JavaScript, CSS и HTML5. SAPUI5 приложения разворачиваются в браузере. В зависимости от устройства приложение (мобильный, планшет или настольного ПК) использует различные библиотеки пользовательского интерфейса.

Используемые библиотеки и приложения хранятся в SAP HCP. Обмен данных происходит через шлюз по протоколу oData с базой данных в облаке HCP.

SAPUI5 использует концепцию Model-View-Controller для представления информации и взаимодействия в интерфейсах (рис. 10).

Каждая из них выполняет следующие функции:

- View - отвечает за определение и визуализации интерфейса.
- Model - управляет данными приложений.
- Controller - реагирует на события и взаимодействие пользователя с интерфейсом путем изменения View и Model. [5]

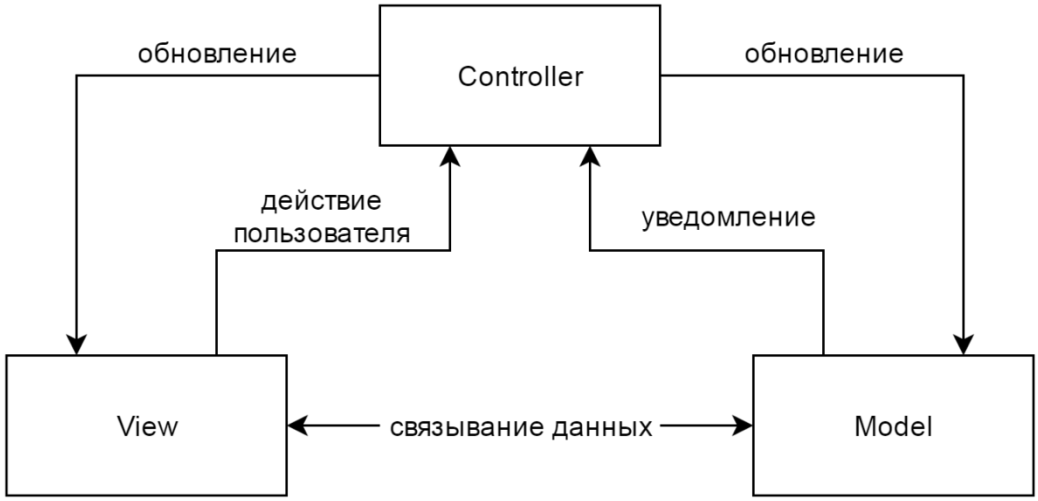


Рис 10. Модель представления данных SAPUI5

3.2 SAP Fiori

Fiori – это интерфейс для основных систем SAP. Это тонкий клиент, написанный на HTML5, не требующий установки новых серверов, новых лицензий от SAP. Может использоваться на смартфоне, планшете, ПК.

SAP Fiori обеспечивает:

- реализацию единообразного представления Web приложений вне зависимости от типа устройства, - представления SAP Fiori практически не отличаются для различных типов устройств с различными форм-факторами,

- ·простой и интуитивно понятный способ работы с приложениями для всех устройств – рабочие станции с веб-браузером, мобильные устройства, как планшеты, так и смартфоны;
- ролевую модель управления доступом к функциям приложений на основе ролей Backend систем SAP,
- использование стандартных механизмов безопасности платформы SAP,
- эффективное управление решением SAP Fiori: приложения могут быть представлены как набором артефактов с общей стартовой страницей, так и представлены по отдельности, как самостоятельные веб-приложения. Приложения могут быть интегрированы в решение SAP NW Portal или в сторонние портальные решения. [10]

3.3 Использование сторонних фреймворков

Последние 10 лет область веб-разработки развивается очень быстрыми темпами и появляются множество библиотек, с помощью которых становится легче и быстрее разрабатывать свой продукт.

Например, библиотека ExtJS написана на JavaScript и работает во всех популярных сейчас браузерах (хотя и с некоторыми различиями), предназначена для создания сложных и насыщенных интерфейсов, которые очень похожи на их аналоги из мира desktop-программ. Она предоставляет разработчику целый набор графических компонентов, от тривиальных кнопок и расширенных элементов обычных HTML-форм, до сложнейших компонентов вроде таблиц, лейаутов и деревьев. Для создания компонентов используется очень простая функция, например следующий код создает панель и показывает на странице:

```
Ext.create("Ext.panel.Panel", {
    height: 100,
    width: 200,
    renderTo: Ext.getBody()
})
```

В библиотеке также есть достаточно много невидимых пользователю компонентов, которые и обеспечивают работу того, что мы видим на экране. Это и получение данных с сервера в фоновом режиме (в формате JSON или XML), обновление частей страницы, локальные хранилища данных, поддержка cookie и многое другое.[8]

Одним из распространенных библиотек является React. Она позволяет создавать интерфейсы. В известном паттерне Model-View-Controller React ближе всего к пользователю. Он отвечает за представление данных, получение и обработку ввода пользователя. Где и в каком виде вы храните

данные, как вы общаетесь с хранилищем, дело ваше. React построен на парадигме реактивного программирования. Этот декларативный подход предлагает описывать данные в виде набора утверждений или формул. Изменение одного из параметров ведёт за собой автоматический пересчёт всех зависимостей. Создание того же панеля с помощью React [9]:

```
ReactDOM.render(
  <div style="height:100px; width:200px;"></div>,
  document.getElementById('root')
);
```

3.4 Использованные инструменты

При разработке интерфейса на SAPUI5 код программы пишется на браузере в специальном редакторе, предоставленном компанией SAP. Разработчику становится неудобно при постоянной проверке проекта в ходе процесса разработки. Функциональность огромная и даже избыточная. Продукт сложен, как большинство продуктов SAP. И SAP Fiori является сложным, требует отдельного изучения способов создания интерфейса на его основе. Сторонние библиотеки хороши тем, что они всегда поддерживаются сообществом программистов и хорошо распространены. Некоторые фреймворки дают готовые компоненты, которые в нашем случае являются избыточными и делают пакет данных более весомыми.

Поэтому, при разработке были использованы локальный сервер Node.js, расположенный на компьютере, которые дает возможность автоматизировать процесс разработки и ускорить процесс промежуточной проверки. И в качестве библиотеки была выбрана jQuery для улучшения читаемости кода и добавления кроссбраузерности. А в остальном программа была написана на чистом JavaScript, HTML5, CSS3.

4 Разработка интерфейса диалога

4.1 Составные части интерфейса системы управления

Как и в общении между людьми, диалог с компьютером должен удовлетворять определенным правилам. Разработка диалога состоит в приведении этих правил в соответствие с психологическими потребностями и представлениями человека.

Диалоговые процессы можно классифицировать по формату поддерживаемых входных сообщений (грамматике) и по способу ведения диалога (управляемые пользователем или системой). Классификацию можно уточнять с учетом вида подсказок, инициализирующих запрос на ввод.

Независимо от грамматики или способа ведения диалога, в его основе лежит следующий цикл:

- явный или неявный запрос на ввод данных;
- ввод данных через процесс ввода;
- проверка входных данных, который повторяется, пока не будут приняты приемлемые входные данные. Если выводится запрос на ввод команды, следующий шаг обработки будет зависеть от введенной команды.

На рис. 11 приведена диаграмма классов интерфейса системы управления ГПС.

Интерфейс объединяет в себе все элементы и компоненты программы, которые способны оказывать влияние на взаимодействие пользователя с ГПС. К этим элементам относятся:

- набор задач пользователя, которые он решает при помощи системы;
- используемая системой метафора и идиомы;
- элементы управления системой;
- навигация между блоками системы;
- визуальный (и не только) дизайн экранов программы;
- средства отображения информации, отображаемая информация и форматы;
- устройства и технологии ввода данных;
- диалоги, взаимодействие и транзакции между пользователем и компьютером. Обратная связь с пользователем;
- порядок использования программы и документация на нее. [2]

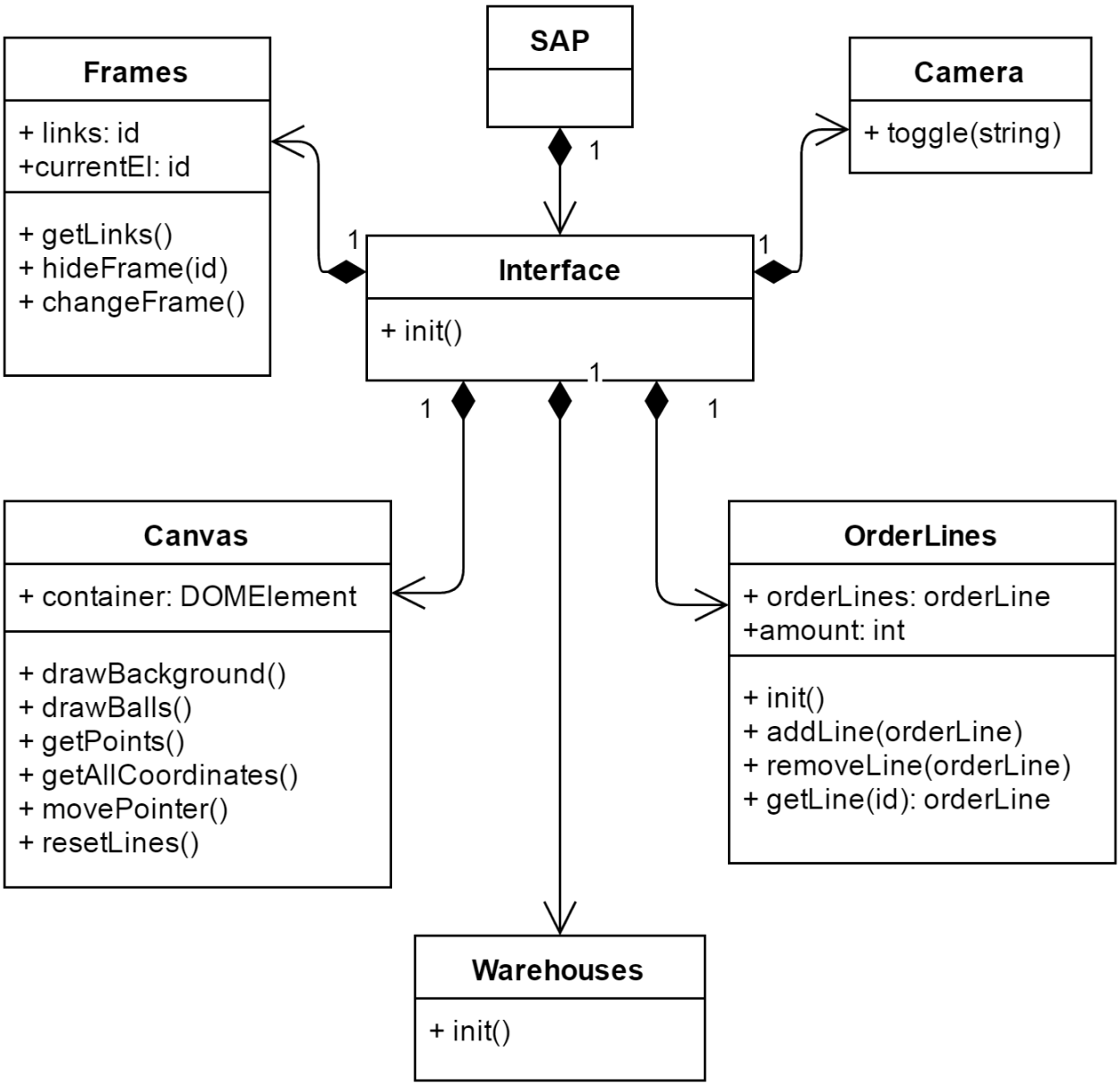


Рис. 11 Диаграмма классов интерфейса

Класс Frames предназначен для управления блоками, которые можно скрывать/раскрывать с помощью показанных в виде меню ссылок (links). Меню основан на идиоматической парадигме, пользователь понимает ее работу на интуитивном уровне. Клик на один из элементов меню вызывает функцию changeFrame, которая в свою очередь скрывает текущий блок и раскрывает указанную.

Класс Canvas выполняет функцию рисования в зависимости переданной команды от пользователя.

Структуру интерфейса можно разделить на два основных компонента:

- процесс диалога, который связывает фоновые процессы в одну систему. Сюда относятся классы Frame, OrderLines, OrderLine, Canvas, Camera, методы которых инкапсулированы от пользователя;

• набор процессов ввода-вывода, которые обеспечивают физическую связь между пользователем и процессом диалога. Набор меню, кнопок, формы и т.д.

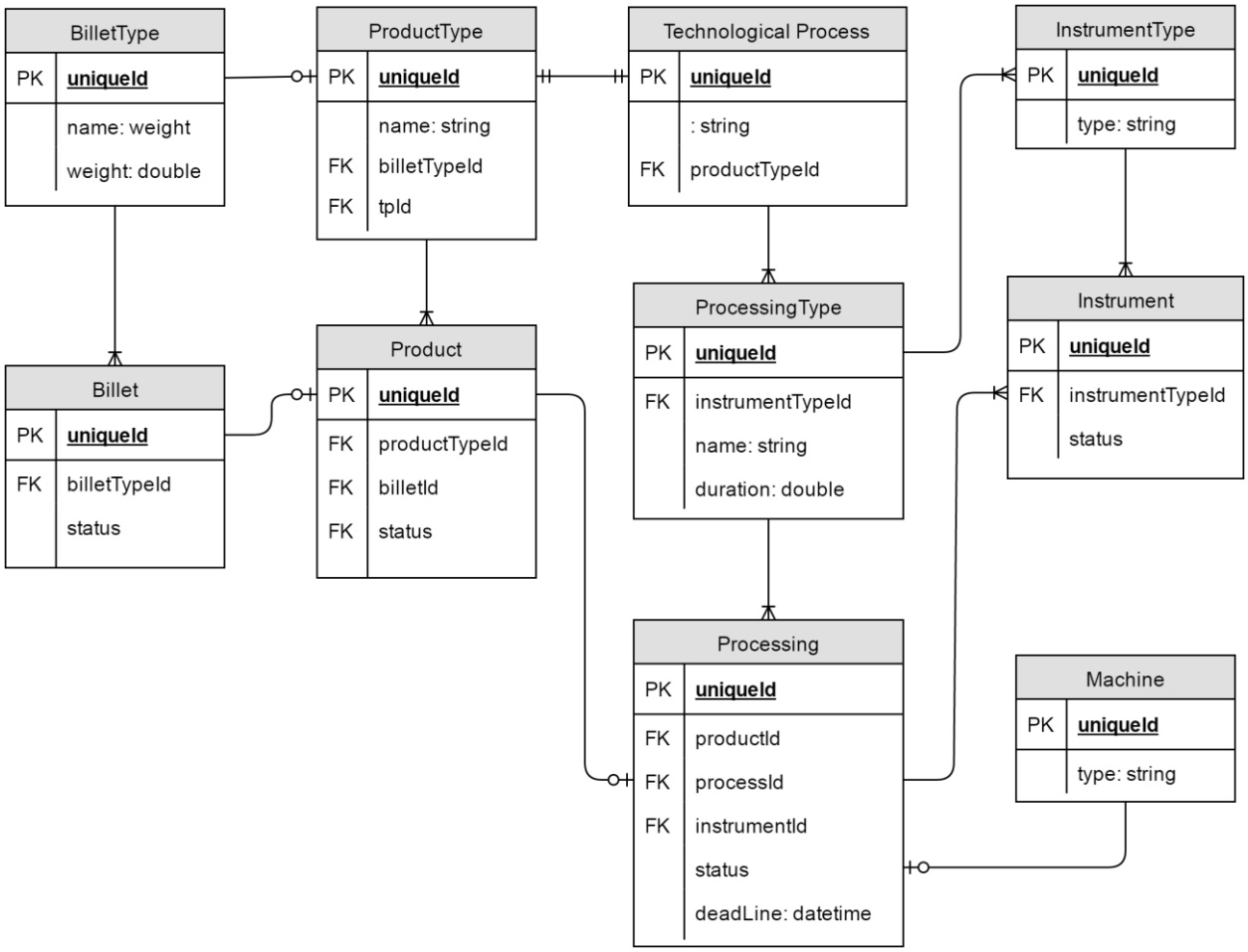


Рис. 12 Информационная модель

4.2 Создание интерфейса управления процессом

Класс OrderLines обертка управляющих команд для диаграммы Ганта. В его обязанности входит первичная инициализация при открытии страницы, добавление новой линии, удаление линии. Заметим что эти функции являются скрытыми от пользователя, так как изменение состава ГПС происходит на более низком уровне и соответственно добавление и удаление линии выполняется изменением кода. Это сопровождается добавлением или удалением соответственно новой таблицы в базу данных SAP HANA. OrderLines не может напрямую воздействовать на базу данных, а являются только контейнерами для нескольких линии, которые создаются как класс OrderLine. Структура этого класса показана на рисунке 13.

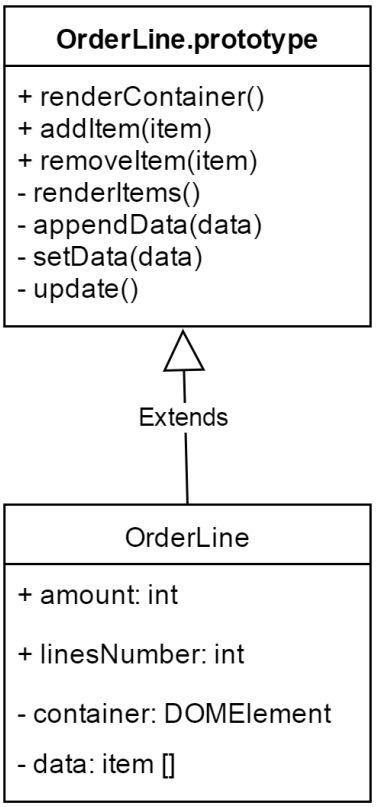


Рис. 13 Структура класса OrderLine

При добавлении новой линии, создается объект типа OrderLine и добавляется в список orderlines объекта OrderLines. С помощью методов этого объекта пользователь сможет добавлять новое изделие или удалять уже существующую. Эти методы вызываются по нажатию соответствующих кнопок. На рис. 14 приведен алгоритм диалога при операции добавления новой детали в очередь, вывод формы добавления данных на дисплей оператора, обработка ошибочного ввода и проверка управляющей команды.

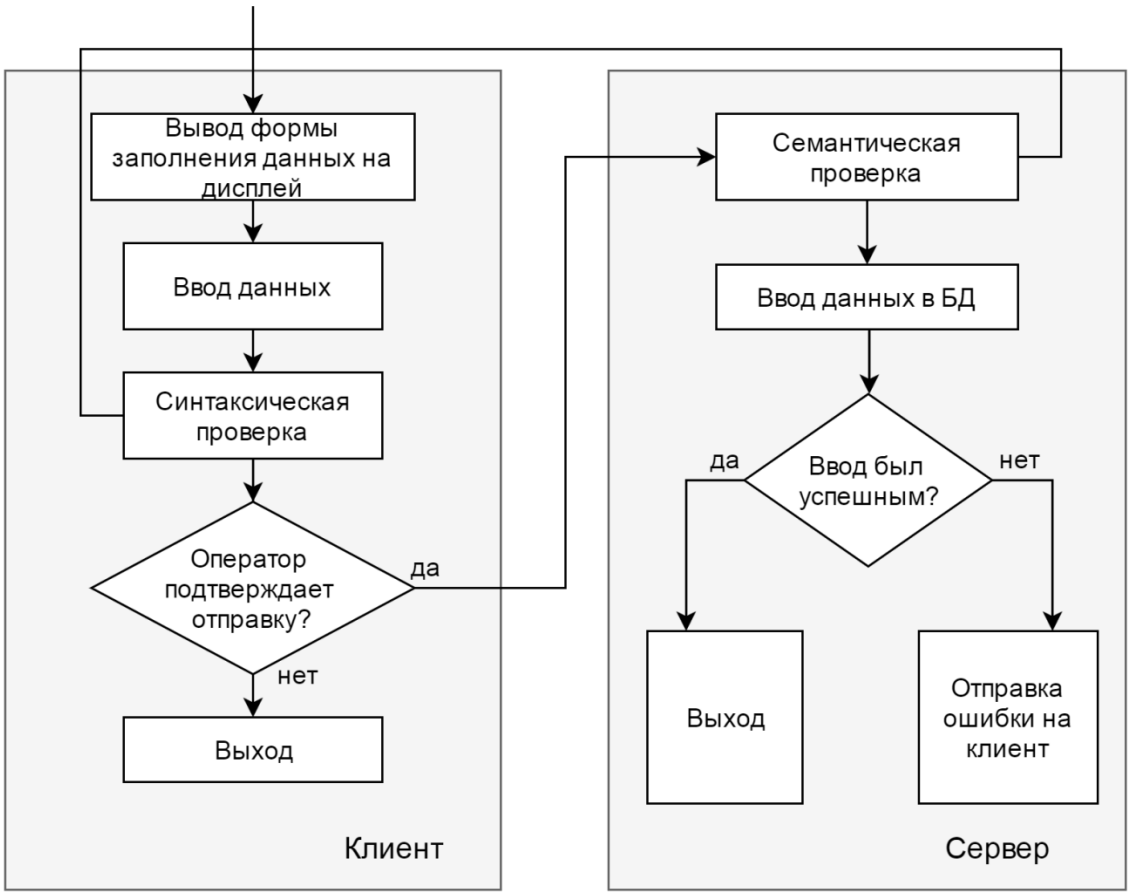


Рис. 14 Алгоритм добавления обработки детали в расписание

Проверка управляющей команды происходит в два этапа: синтаксической и семантической. После семантической проверки управляющей команды новые данные будут добавлены в соответствующие таблицы на базе данных. Если пользователь собирается добавить новый тип детали, ему будет предложено указать список необходимых обработок.

Внешний вид формы управления линией приведена на рисунке 15.

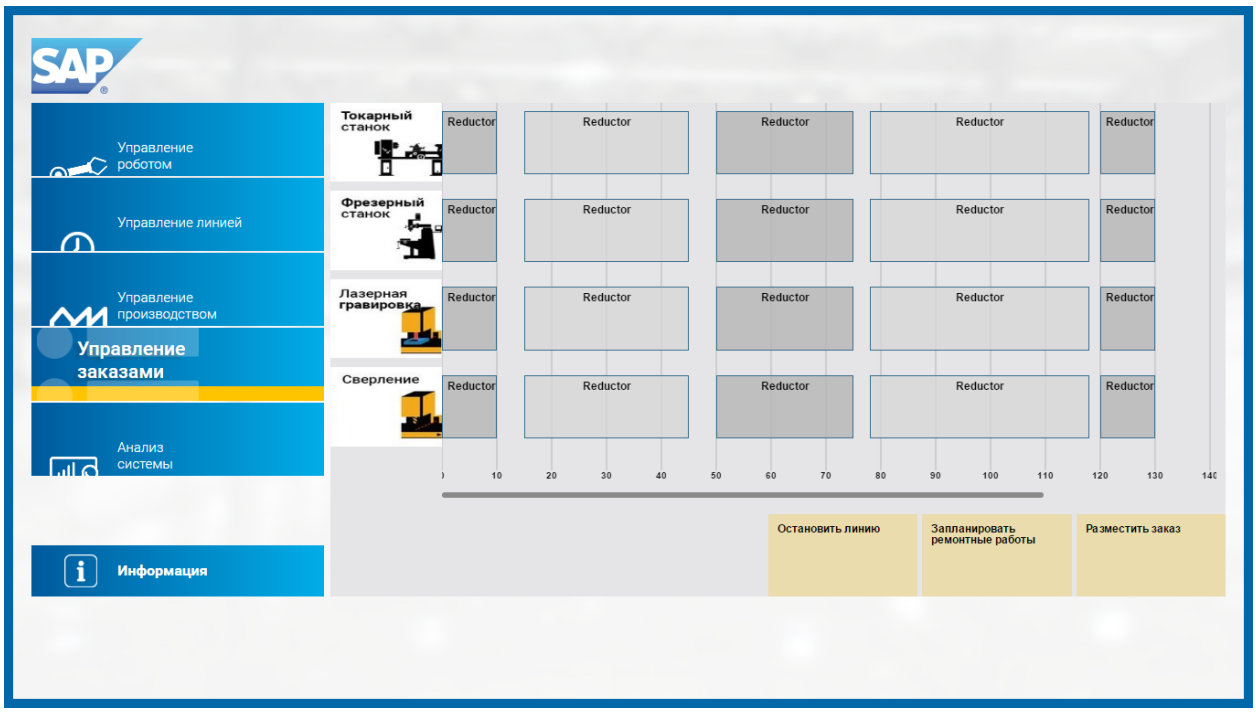


Рис. 15 Общий вид интерфейса диаграммы Ганта

3.3 Создание интерфейса управления складом

Класс Warehouses инициализирует компоненты, предназначенные для управления складом. В нашем случае создаются сразу 2 компонента для управления складом заготовки и складом готовых деталей. Компоненты имеют тип Warehouse, который предоставляет функцию addCell, с помощью которого при инициализации на склад добавляются ячейки типа Cell. Класс Cell предоставляет функции putDetail и takeDetail. По свойству isEmpty узнаем состояние ячейки. Диаграмма классов компонентов склада показана на рисунке 16.

Внешний вид формы управления складом показана на рис. 17. На рисунке можем увидеть кнопки управления штабелером, которые вызывают функции у объектов управления складом.

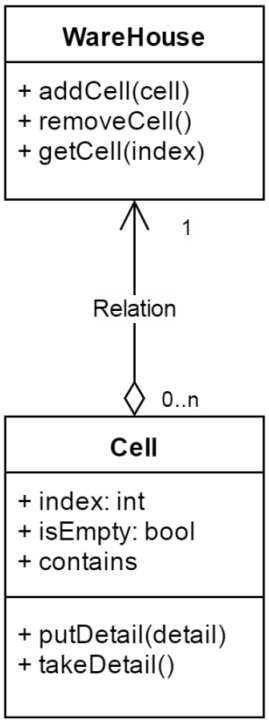


Рис. 16 Структура компонентов склада

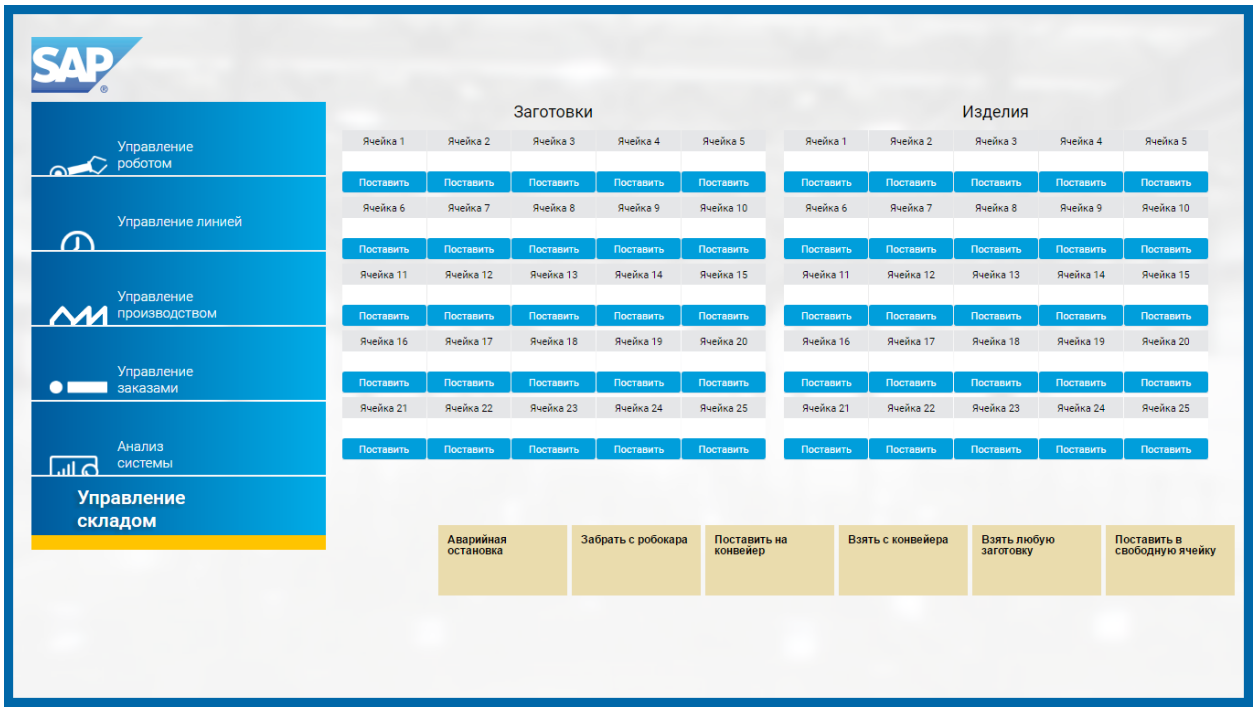


Рис. 17 Общий вид интерфейса управления складом

Заключение

Быстрое распространение интернета, увеличение количества пользователей привело к бурному развитию интернет технологии. В том числе и облачных технологии, с помощью которых компании начинают автоматизировать бизнес процессы. Появление платформы SAP HANA с базой данных хранящей данные на оперативной памяти дало возможность управлять системами, работающими на реальном времени удаленно. И параллельно с интернетом развиваются быстро и браузеры, что дает разворачивать сложные программы прямо в браузере. И это в сумме дает гибкую систему, который может управлять даже производственным процессом, анализировать ее состояние, и т.д.

На данном курсовом проекте:

- проводился обзор технологиям компании SAP, таких как SAP HANA Index Server, SAP XS, SAPUI5 и возможность их расширения с помощью интернет технологии;
- проводился обзор инструментов использующихся для создания интерфейсов;
- разрабатывался интерфейс для системы управления ГПС, предназначенный для наблюдения за ходом производственного процесса с помощью диаграммы Ганта и возможностью добавления и удаления новых изделии;
- разрабатывался интерфейс, предназначенный для управления складом.

Список литературы

1. Оперативное управление в ГПС /В.Ф. Горнев, В.В. Емельянов, М.В. Овсянников. – М.: Машиностроение 1990 г.
2. Технология разработки прикладного программного обеспечения /Соловьев С. В., Цой Р. И., Гринкруг Л. С. – 2011 г.
3. SAP Community Network.[Электронный ресурс]: http://scn.sap.com/
4. SAP HANA Cloud Documentation. [Электронный ресурс]: https://help.hana.ondemand.com
5. Обзор элементов фреймворка SAPUI5. [Электронный ресурс]: https://sapui5.netweaver.ondemand.com/sdk/explored.html
6. Портал обучения инструментам SAP [Электронный ресурс]: http://saphanatutorial.com/
7. Википедия [Электронный ресурс]: https://ru.wikipedia.org/
8. ExtJS [Электронный ресурс]: https://www.sencha.com
9. ReactJS [Электронный ресурс]: https://facebook.github.io/react/
10. Sap Fiori [Электронный ресурс]: http://sapland.ru/